Final Project

m11415067 周珥豪

Data Cache Prefetching Using a Global History Buffer

Kyle J. Nesbit and James E. Smith
Department of Electrical and Computer Engineering
University of Wisconsin - Madison
{nesbit, jes}@ece.wisc.edu

10th International Symposium on High Performance Computer Architecture (HPCA'04)

Outline

1. Implementation
2. Performance Evaluation
3. Conclusion

Implementation

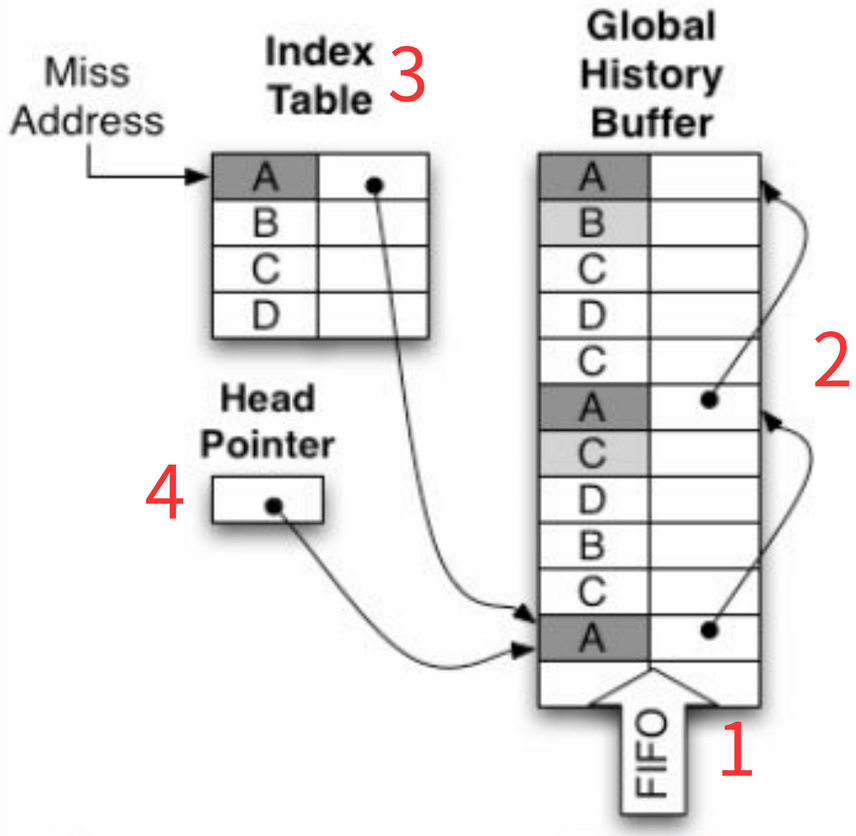


Figure 5: GHB Global / Address Correlation

Implementation Structure

- In sim-outorder.c
  - Index Table
    - pc_tag
    - head
    - head_stamp
    - valid
  - Global History Buffer
    - block_addr
    - prev
    - stamp

```c
struct ghb_entry_t {
  md_addr_t block_addr;
  int prev;
  unsigned int stamp;
};

struct it_entry_t {
  md_addr_t pc_tag;
  int head;
  unsigned int head_stamp;
  int valid;
};
```

Implementation Function

- In sim-outorder.c

- ghb_train()
  - IT lookup
    - Locate per-PC history chain
  - append to GHB(ring)
  - renew IT
    - So that next time you can find the latest header with your PC
  - Stride detection
  - Prefetch issuer
    - This will follow cache's miss/fill/replacement policy

Implementation Function

- In sim-outorder.c

```
ghb[ghb_head].block_addr = block_addr;
ghb[ghb_head].prev = prev;
ghb[ghb_head].stamp = ghb_stamp++;

itab[it_idx].valid = 1;
itab[it_idx].pc_tag = pc;
itab[it_idx].head = ghb_head;
itab[it_idx].head_stamp = ghb[ghb_head].stamp;
```

```
if (s1 != 0 && s1 == s2) {
  #if 1
  if (cache_dl2) {
    int issued = 0;

    int k;
    for (k = 1; k <= ghb_degree; k++) {
      md_addr_t pf_addr_k = (md_addr_t)(a0 + (md_addr_t)k * s1);
      ghb_pref_candidates++;

      (void)cache_access(cache_dl2, Read, pf_addr_k, NULL,
                         1, pt_cycle, NULL, NULL);
      ghb_pref_issued++;
      issued++;
    }
    ghb_stride_found++;
  } else {
    ghb_pref_skip_no_ul2++;
  }
  #endif
}
```

Implementation

Verify

```
ghb_trains                    0 # GHB updates on ul2 demand misses
it_hits                       0 # Index table hits
it_misses                     0 # Index table misses
ghb_overwrites                0 # GHB ring overwrites
ghb_hook_misses               0 # number of ul2 misses observed by local GHB hook (store path)
ghb_lookups                   0 # number of stride lookups after training (observe-only)
ghb_chain2_ok                 0 # number of times the per-PC chain depth >= 2
ghb_stride_found              0 # number of constant strides detected (observe-only)
ghb_pref_candidates           0 # number of stride-based prefetch candidates (not issued)
ghb_pref_issued               0 # number of stride-based prefetches issued to ul2
ghb_pref_skip_no_ul2          0 # number of times prefetch skipped because ul2 disabled
```

No GHB

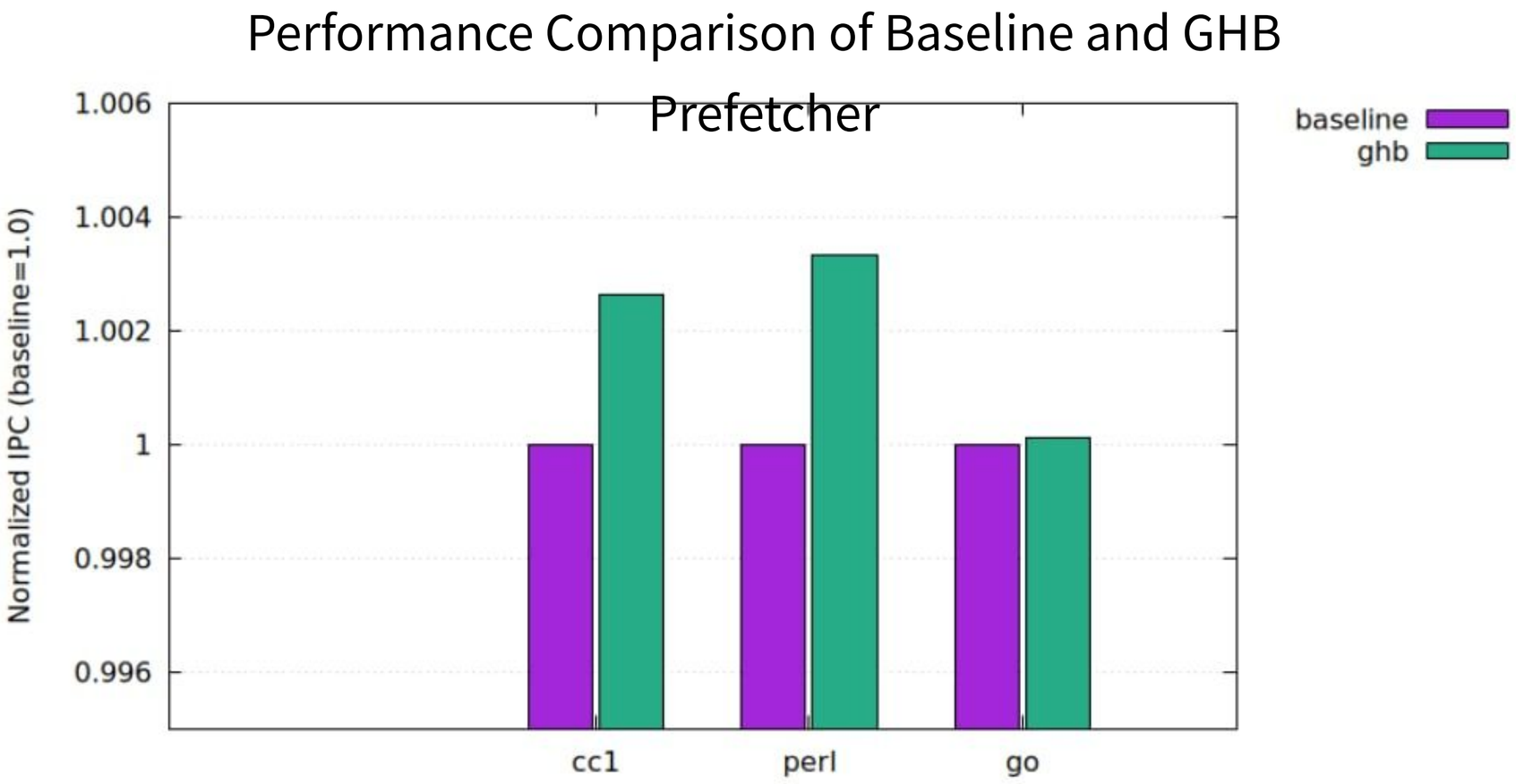
```
ghb_trains                  530 # GHB updates on ul2 demand misses
it_hits                     491 # Index table hits
it_misses                    39 # Index table misses
ghb_overwrites                0 # GHB ring overwrites
ghb_hook_misses             530 # number of ul2 misses observed by local GHB hook (store path)
ghb_lookups                 530 # number of stride lookups after training (observe-only)
ghb_chain2_ok               479 # number of times the per-PC chain depth >= 2
ghb_stride_found            159 # number of constant strides detected (observe-only)
ghb_pref_candidates         636 # number of stride-based prefetch candidates (not issued)
ghb_pref_issued             636 # number of stride-based prefetches issued to ul2
ghb_pref_skip_no_ul2          0 # number of times prefetch skipped because ul2 disabled
```

Have GHB

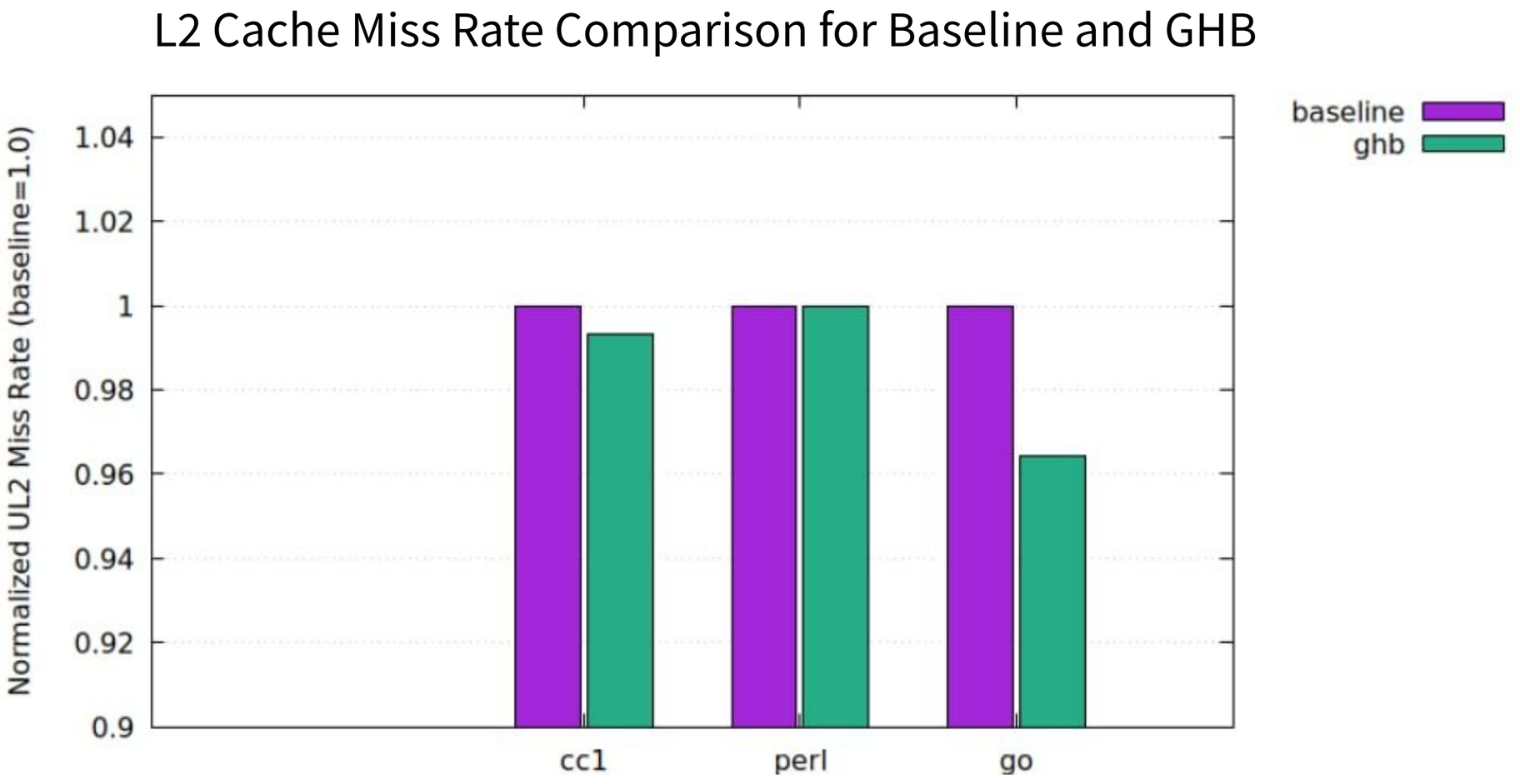
Performance Evaluation

- Simplescalar 3.0
- New Index Table & GHB
- Benchmarks
  - cc1
  - perl
  - go
- Evaluation Focus
  - Performance Comparison of Baseline and GHB Prefetcher
  - L2 Cache Miss Rate Comparison for Baseline and GHB
  - Prefetch Traffic vs. GHB Prefetch Degree
  - Delta UL2 Miss Rate vs. Degree

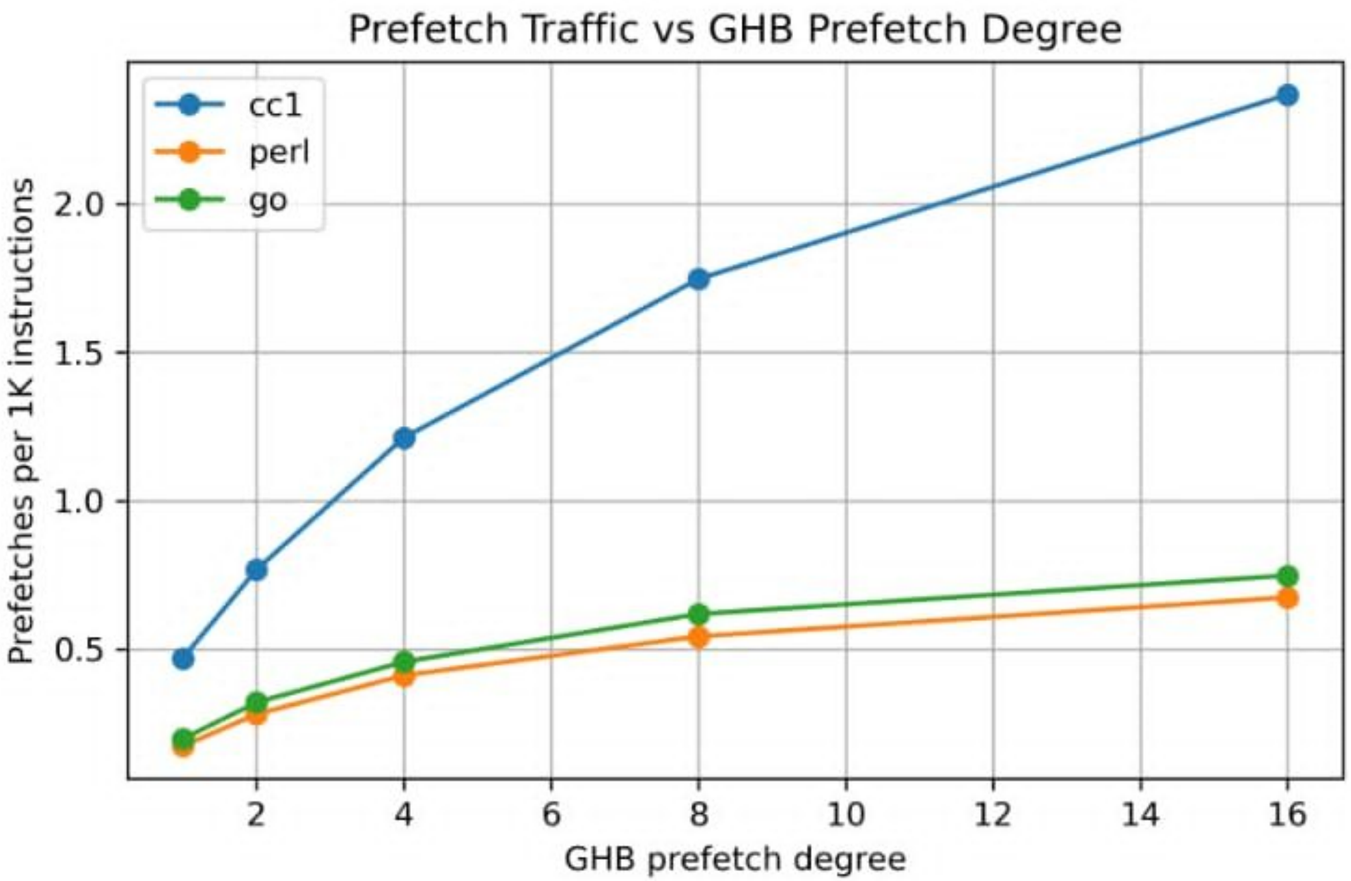
Performance Evaluation



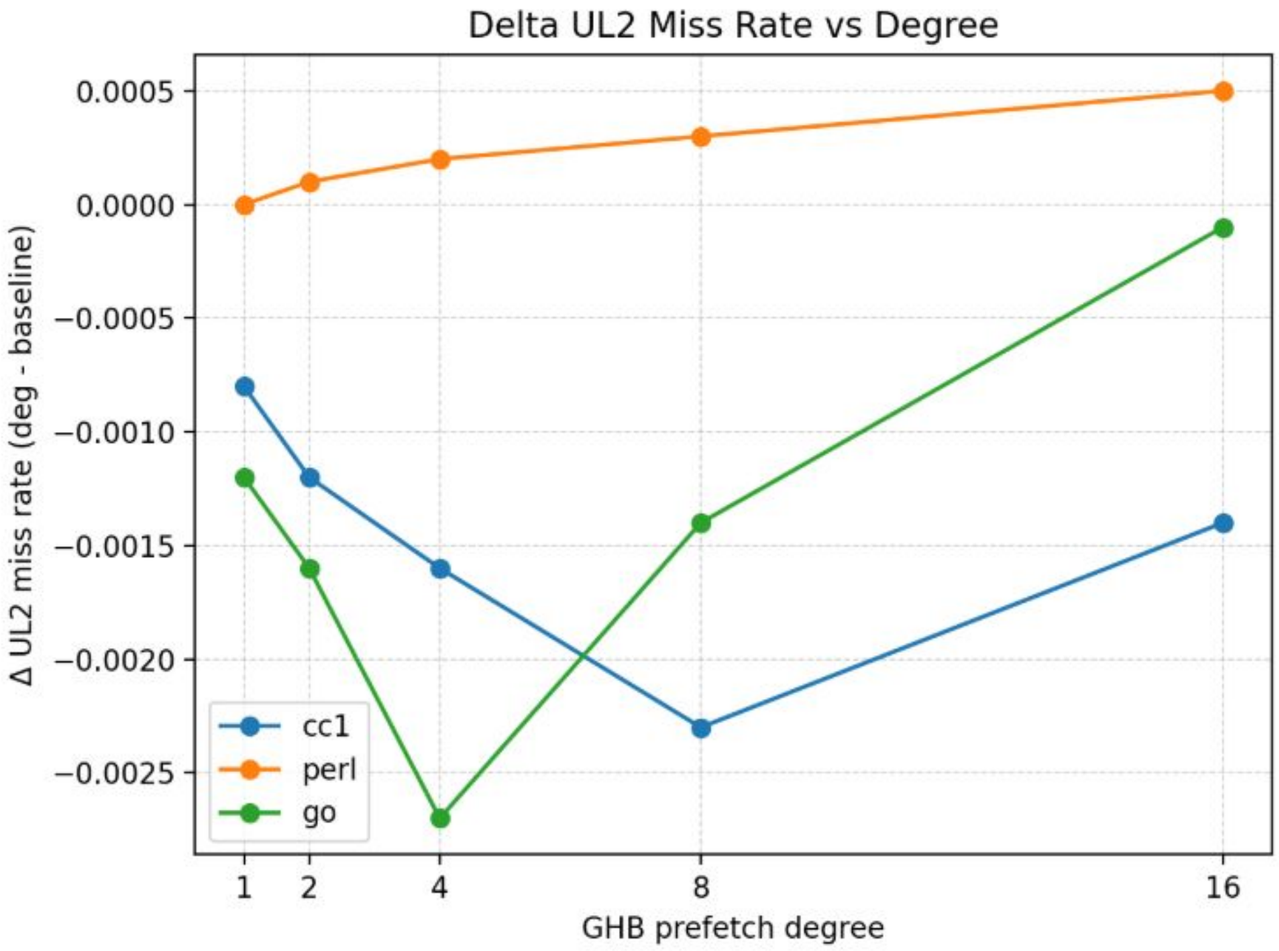
Performance Evaluation



Performance Evaluation



Performance Evaluation



Conclusion

- GHB Pros
  1. reduce stale data
  2. Preserve complete global history
  3. Less hardware requirements
- GHB Cons

  → Requires multiple table accesses
- Additional Contribution

  → The GHB architecture provides a unified prefetching structure.